



Python for Fixed Income

COURSE 2 · BOND MATH

Case Study: Convexity and the September Book

WEEK 2 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Week 2 ends: one measure, one real job

Hands-on, 10.1 to 10.4: convexity

The four bonds of the week, settled 30 November 2026. Where duration's straight line misses, the convexity function, a check with three Excel PRICE cells, and a full reprice.

Case study, 10.5 to 10.10: the September book

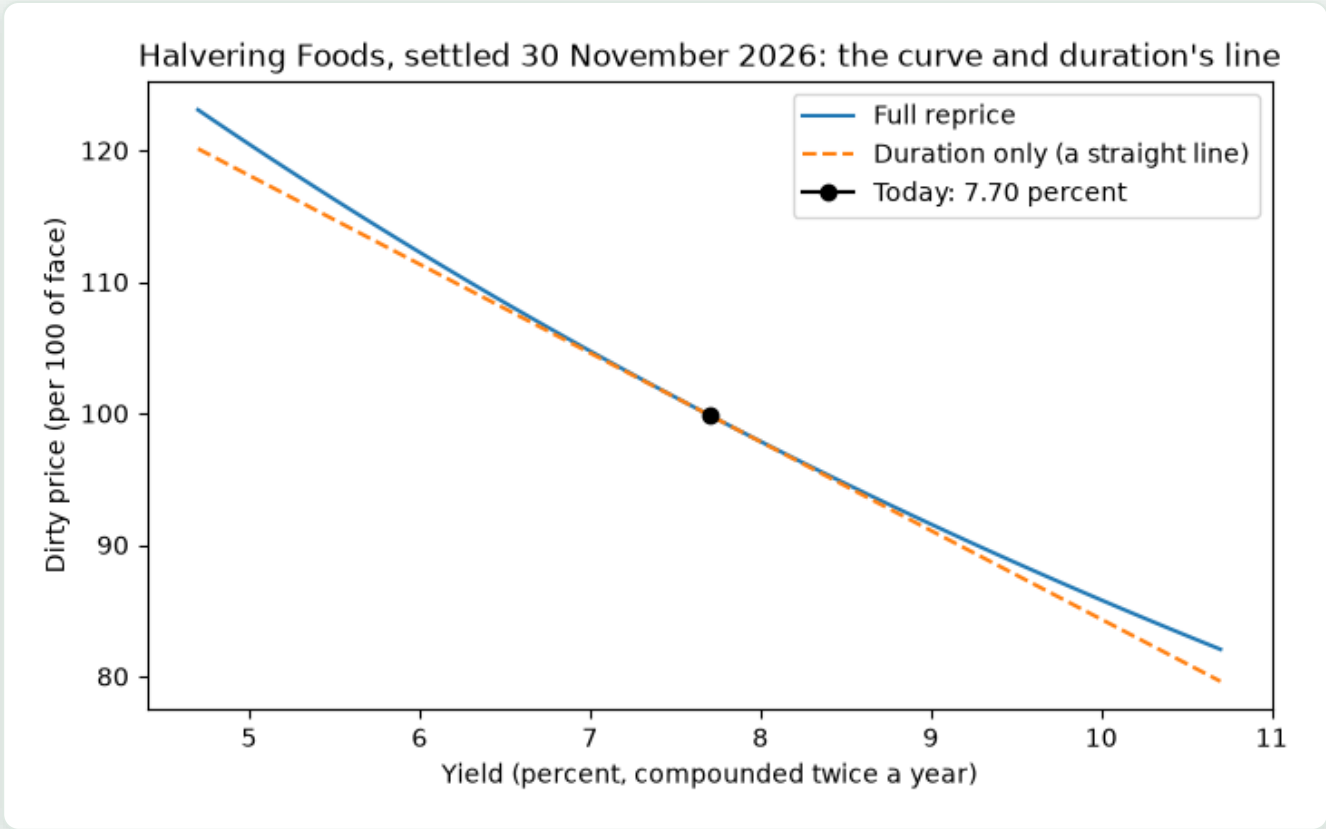
holdings.csv, 200 bonds. Price, accrued, yield, and duration rebuilt for every bond and checked with assert. Then the book's DV01 in dollars.

Week 2: dates, accrued, and risk

Day 6	Dates and coupon schedules
Day 7	Day counts and accrued interest
Day 8	Clean and dirty price
Day 9	Duration and DV01
Day 10	Convexity and the September book

Invented issuers and numbers. The day describes the book and never says a bond is cheap or rich.

Where duration's straight line misses



Halvering, per 100 of face

Shift, bp	Full	Duration	Miss
-300	23.2141	20.2534	2.9607
-100	7.0585	6.7511	0.3073
+100	-6.4633	-6.7511	0.2879
+300	-17.8202	-20.2534	2.4333

The miss is positive both ways, and three times the move is about nine times the miss.

Duration is a straight line laid against a curve. It works for small moves, not large ones.

Convexity: the bend, in years squared

```
# dP / P, with dy as a decimal
-modified_duration * dy + 0.5 * convexity * dy ** 2
```

Each cash flow adds $CF \times t \times (t + 1/f) / (1 + y/f)^{(ft + 2)}$, and the sum is divided by the dirty price. Roughly time squared: long bonds have far more convexity.

Other sources scale it differently: divided by 100, or halved. The course's number is unscaled. Halvering: 59.5474.

Bond	Mod. duration	Convexity
Pellwick	2.6051	8.3305
Quorvane	4.1804	20.8885
Dunmarrow	5.4623	36.3887
Halvering	6.7598	59.5474

Halvering against Pellwick: duration 2.6 times larger, convexity 7.1 times.

Units: years squared. A bullet bond's convexity is always positive.

Excel has no convexity function: three PRICE cells

Price the bond at today's yield, 1 bp lower, and 1 bp higher. The curvature, over the dirty price, is a central difference. Accrued cancels out.

Excel side by side (tested in Excel)

A1	=PRICE(DATE(2026,11,30),DATE(2036,9,30),7.5/100,7.7/100,100,2,0)	98.622491
A2	=PRICE(...,7.7/100-0.0001,100,2,0)	98.690032
A3	=PRICE(...,7.7/100+0.0001,100,2,0)	98.555009
A4	=100*7.5/100/2*COUPDAYBS(...)/COUPDAYS(...)	1.250000
A5	=(A2+A3-2*A1)/((A1+A4)*0.0001^2)	59.547369

59.5474

Halvering, bondmath.convexity

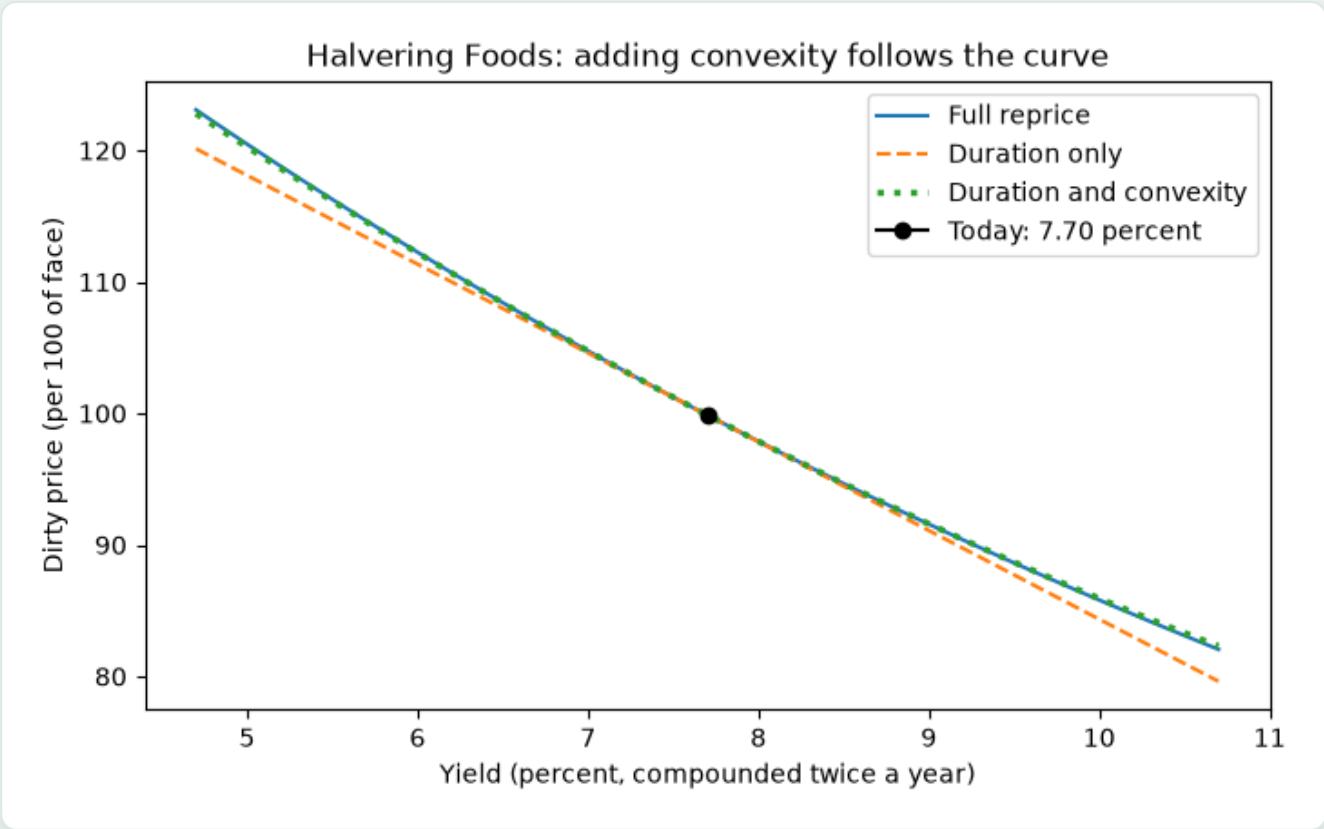
4.8e-6

gap to Excel's central difference

28 passed

with the new test: every reference row, 1e-3

Duration plus convexity against a full reprice



Halvering, miss per 100 of face

Shift, bp	Duration only	With convexity
-300	2.9607	0.2845
-100	0.3073	0.0100
+100	0.2879	-0.0095
+300	2.4333	-0.2430

With convexity the miss is small and changes sign. The full reprice is the answer.

+100 bp, Halvering: estimate -6.4620 percent, full reprice -6.4715.

The September book: check it, then add it up

Context

holdings.csv: 200 invented bullets as of 30 September 2026, semiannual, 30/360. Its price, accrued, yield, and duration are rounded to 3 decimals.

Objective, with your own module

1. price from yield_pct
2. accrued from the dates
3. yield_pct from price
4. duration, and a new convexity column
5. the benchmark, and what does not match
6. book DV01 in dollars, and its duration

Tolerances, with reasons

Column	Tolerance
price, accrued, duration	0.0005 + 1e-9
yield from rounded price	0.001 percent

Rounded to 3 decimals: off by up to 0.0005. A yield solved from a rounded price inherits it.

Two Predict questions. Every check is an assert: one bond out, and the notebook stops.

Price and accrued, every bond

```
# one bond at a time, with zip
book["price_calc"] = [bondmath.price(s, m, c, y)
    for s, m, c, y in zip(...)]
assert price_gap.max() <= ROUNDED_3
```

0.0004985

largest price gap, per 100

0.0005

largest accrued gap, per 100

All 200 bonds pass both checks.

Why the extra 1e-9

Bond 99113JAA2: the module's accrued is 0.0875, a rounding tie. The file holds 0.087. The true gap is exactly 0.0005.

0.00050000000000000004

Floating point stores neither number exactly. A tolerance of exactly 0.0005 would fail a bond that is right.

Yield from the file's price

Now the other way round: solve each yield from the file's rounded price with `yield_to_maturity`, and compare with `yield_pct`.

```
book["yield_calc"] = [
    bondmath.yield_to_maturity(s, m, c, p)
    for s, m, c, p in zip(...)]
```

Every price carries a rounding error of up to 0.0005 per 100. Where does it hurt the yield most?

Predict 1

Solved back into a yield, which bonds miss the file's yield by the most?

- A) The longest bonds: the most cash flows
- B) No pattern: rounding errors are random
- C) The shortest: small duration, large yield error

Short bonds miss most; two bonds disagree

Predict 1: C, the shortest bonds

Bond	Mod. duration	Yield gap, %
99006GAC4	0.9271	0.000440
99026AAD1	1.0050	0.000433
99003DAG5	1.0169	0.000424

Average gap: 36 bonds under 3 years, 0.000139 percent; 79 bonds over 7 years, 0.000027 percent.

Duration: largest gap 0.0004982, all 200 pass. Then a convexity column, checked against Excel's central difference.

The benchmark: 821 bonds, two outside

CUSIP	Accrued gap	Duration gap
99018SAA8	0.0289	0.0052
99128YAE4	0.0523	0.0056

Both mature on 28 February of a non-leap year. Excel's month-end rule puts coupons on 31 August; the data generator kept the 28th.

Both are conventions. Left out by name; a new failure still stops.

Book DV01 in dollars, and the book's duration

```
# dollars per bp, each position
dv01(...) * par_held / 100
# weights: market value, dirty price
(weights * duration_calc).sum()
```

306,996.11

book DV01, dollars per bp

6.2548

weighted duration, years

Face 489,000,000 dollars, market value 490,819,215.00 dollars, equal to the file's total to the cent.

Predict 2

Book duration x market value x 0.0001. How does it compare with the book DV01?

- A) About 6 times larger: duration is in years
- B) The same, up to the rounding of the prices
- C) Much smaller: the weights add up to 1

One number in two units

306,996.21

duration x MV x 0.0001

306,996.11

book DV01

-0.10

difference, dollars

Each position's DV01 is duration x market value x 0.0001. Summed, the market values become weights. Years, or dollars per bp: one number.

Observations

- Every holdings bond reproduces at the course's tolerances; the largest gaps are rounding.
- Yields solved from rounded prices miss most on the shortest bonds.
- In the benchmark, only the two 28 February maturities differ: two conventions.

These describe the file and the book. None says which bond to buy or sell.

Today's Excel, every formula tested

Job	Excel (tested)	Returns
Convexity	<code>= (A2+A3-2*A1) / ((A1+A4)*0.0001^2)</code>	59.547369
+100 bp estimate	<code>= (-MDURATION(...)*0.01+0.5*A5*0.01^2)*100</code>	-6.462029
+100 bp reprice	<code>= (PRICE(...,7.7/100+0.01,...)-A1)/(A1+A4)*100</code>	-6.471508
Book DV01	<code>=SUMPRODUCT(E2:E201,H2:H201)/100</code>	306,996.11
Market value	<code>=SUM(I2:I201)</code>	490,819,215.00
Weighted duration	<code>=SUMPRODUCT(I2:I201,J2:J201)/SUM(I2:I201)</code>	6.254772

The book sheet: one bond a row; H holds each DV01 per 100, I market value, J MDURATION. SUMPRODUCT multiplies two columns row by row and adds.

Excel's totals equal the module's. A1 to A5 are in section 10.3, A6 and A8 in 10.4.

End of week 2: tag it, compare it

Step	Command	What you see
1	git status --short, git diff --stat	M bondmath.py, M test_bondmath.py
2	git add, git commit	convexity and its test, committed
3	git tag week2, git tag	week1, week2
4	git log --oneline --decorate	tag: week1 and tag: week2
5	git diff --stat week1 week2	304 and 155 lines added

Two tags name two commits. diff between them is everything week 2 added.

The notebook's work folder is new, so the setup rebuilds the week1 commit: each file cut just before add_months.

Bring it home: paste, test, commit, tag

1

Your folder

Set-Location to my_bondmath, in your projects folder.

2

Paste

convexity at the end of bondmath.py, its test at the end of test_bondmath.py.

3

Test

python -m pytest -q: 28 passed, or more with your exercises.

4

Commit

git diff --stat, git add, git commit with a subject and a body.

5

Tag

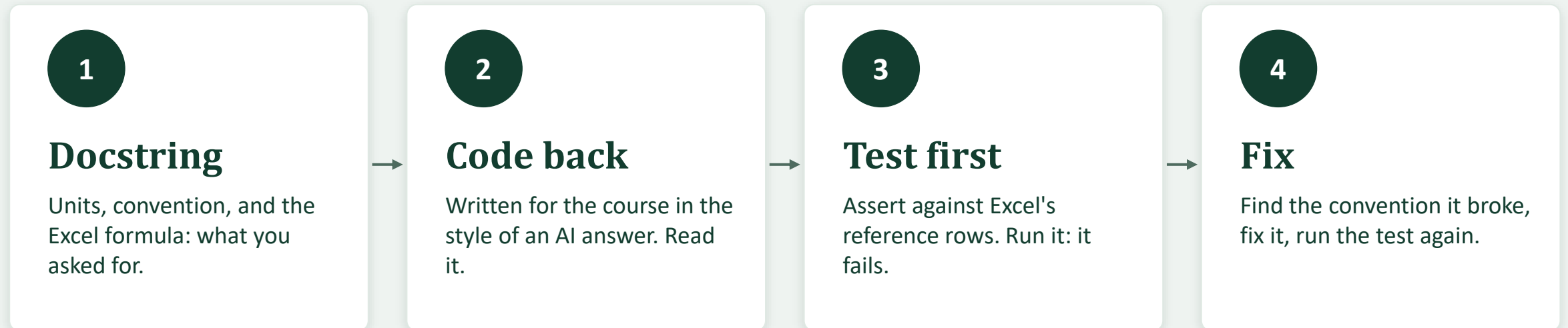
git tag week2, git log --oneline --decorate, git diff --stat week1 week2.

Your folder has had its week1 tag since day 5. Copy fallen behind? Take the start-of-day files from today's setup cell.

In Colab: download bondmath.py and test_bondmath.py from the work folder under /tmp.

The AI check

The docstring is the prompt. The code that comes back runs, and has one convention error.



Today: convexity left in periods squared, never divided by the frequency squared: four times too large.

Code that runs is not the same as code that is right.

What you can do now

- Say where duration's straight line misses, and why
- Compute convexity, in years squared, unscaled
- Check it with three PRICE cells, and estimate with both terms
- Reproduce a book's columns with assert, and add up its DV01
- Tag the end of a week, and compare two tags

Week 3, Day 1 Treasury curve interpolation: a yield split into curve and spread